

Telecom Operations Agent

Hands-on Agentic AI Problem Statement

Objective: Build a multi-agent telecom operations assistant using LangChain/LangGraph, tool calling, and LangSmith observability.

1. Business Scenario

A telecom company operates a Network Operations Center (NOC) that handles customer issues, network health checks, billing questions, and network incidents. Today, engineers manually check multiple backend systems to investigate a request. The company wants an AI-powered agent that can understand a request, decide what information is required, call the right backend tools, interpret the results, and provide a clear final response.

The solution should demonstrate **agentic behavior**: the LLM should decide which action is required rather than the workflow being hard-coded for every question.

2. Agents to Build

Agent	Responsibility	Typical Actions
Telecom Service Agent	Primary customer-facing agent. Understands the user's request and coordinates the appropriate specialist.	Classify request, call specialist agents/tools, combine results, provide final response.
Network Operations Agent	Handles network and tower-related investigation.	Check regional network status; check tower health; inspect power, temperature, capacity and connections.
Customer Support Agent	Handles customer account and billing requests.	Look up customer; retrieve plan; calculate bill and additional charges.
Network Diagnostics Agent	Investigates connectivity and network faults.	Run ping/connectivity tests; interpret latency and packet loss; determine whether a fault is confirmed.
Incident Management Agent	Handles confirmed network incidents.	Create incident tickets with severity, description and affected region.

3. Available Backend Tools

Network Tools

- `check_network_status(region)` — returns regional network condition.
- `check_tower_health(tower_id)` — returns tower location, power status, temperature, active connections, maximum capacity and overall status.

Customer Tools

- **lookup_customer(customer_id)** — returns customer, plan and account information.
- **calculate_bill(customer_id, extra_charges)** — calculates the customer's total bill.

Diagnostic Tools

- **run_ping_test(target_ip)** — returns latency, packet loss and connectivity status.

Incident Tools

- **create_incident_ticket(severity, description, affected_region)** — creates a simulated incident and returns a ticket ID.

4. Agent Behavior Requirements

The most important requirement is that the system should not blindly execute a predefined sequence. The LLM should use the user's request, conversation context and available tool definitions to determine the next action.

The agent should be capable of the following loop:

User Request → LLM → Tool/Agent Decision → Execute → Result → LLM → Another Tool/Agent if required → Final Response

For a simple request, one tool call may be sufficient. For a complex network incident, the agent may need several tool calls and specialist agents before it can answer.

5. Core Challenge Scenarios

Scenario 1 — Regional Network Status

User asks: *"What is the current network status in the South region?"*

Expected behavior: Telecom Service Agent identifies this as a network request and routes it to the Network Operations Agent, which checks the regional network status and returns the result.

Scenario 2 — Customer and Billing

User asks: *"Look up customer CUST003 and calculate their total bill with an additional charge of 150 rupees."*

Expected behavior: Customer Support Agent retrieves the customer information and then calculates the bill using the appropriate inputs.

Scenario 3 — Network Fault Investigation

User asks: *"Multiple users in Bangalore East are reporting no service. Tower TWR-003 might be down. Please investigate and create an incident if necessary."*

Expected behavior: The agent should investigate the issue, potentially using connectivity diagnostics and tower health information. If the evidence confirms a fault, the Incident Management Agent should create an incident ticket and return the ticket ID.

Scenario 4 — General Knowledge

User asks: *"What is the difference between 4G and 5G?"*

Expected behavior: The system should recognize that no telecom backend tool is required and answer directly.

6. Detailed Multi-Agent Scenario

Use the following scenario as the main demonstration:

User Request

"Multiple users in Bangalore East are reporting complete loss of service. Tower TWR-003 may be down. Please investigate the problem and create an incident if the network fault is confirmed."

A possible execution path is:

1. **Telecom Service Agent** receives the request.
2. It determines that this is a network fault rather than a billing/customer request.
3. **Network Operations Agent** checks TWR-003 health.
4. **Network Diagnostics Agent** runs a connectivity test against the tower/network endpoint.
5. The diagnostic results are returned to the appropriate agent.
6. The agent evaluates whether the evidence indicates a confirmed fault.
7. If confirmed, **Incident Management Agent** creates an incident ticket.
8. The **Telecom Service Agent** summarizes the investigation, evidence and ticket information for the user.

Important: the exact sequence should be driven by the agent's decisions and available information, not unnecessarily hard-coded.

7. LangGraph Implementation Requirement

Implement the workflow using LangGraph. The graph should support repeated LLM/tool interactions until the agent has enough information to provide a final response.

```
START
↓
Telecom Service Agent
↓
Specialist Agent / Tool
↓
Tool Result
↓
LLM decides next action
↓
Repeat if required
↓
Final Response
↓
END
```

8. Tool Calling Visibility

During the demonstration, make the agent execution observable. Show what the LLM receives at the first call, including the user message and the available structured tool definitions. Then show the model's tool-call decision, tool arguments, tool result, subsequent LLM call, and final response.

The audience should be able to clearly see the difference between **LLM decision-making** and **tool execution**.

9. LangSmith Observability

Integrate LangSmith tracing so that the complete execution can be inspected. The trace should make it possible to identify the user input, agent/LLM calls, tool calls, tool arguments, tool outputs, iterations, latency, token

usage where available, and final response.

10. Minimum Deliverables

1. A working Telecom Service Agent.
2. At least three specialist agents from the defined agent list.
3. The backend tools described in this problem statement.
4. A LangGraph workflow supporting iterative agent/tool execution.
5. At least four demonstration scenarios.
6. LangSmith tracing for the agent workflow.
7. Clear console output showing the execution sequence.
8. A final summary showing which agents/tools were used and why.

11. What You Should Be Able to Explain

- How the LLM decides whether a tool or specialist agent is required.
- How tool definitions are made available to the LLM.
- What the model returns when it decides to call a tool.
- How LangGraph routes the tool call to the appropriate node.
- How tool results are returned to the LLM.
- How multiple agent/tool calls can form an agentic workflow.
- How LangSmith helps trace and debug the complete execution.

Final Objective

Build a telecom operations assistant that behaves like a real NOC support system: it understands the request, chooses the right specialist capability, uses tools when needed, combines evidence, and responds with a useful business outcome.